

**Professor**

Prof. Dr. Volker Roth  
volker.roth@unibas.ch

**Teaching Assistants**

giovanni.abati@unibas.ch  
manuel.jahn@unibas.ch  
kumar.shashwat@stud.unibas.ch

**Institute**

Department of Mathematics  
and Computer Science  
Spiegelgasse 1, 4051 Basel

## Exercise 7

Due date: **Tuesday, April 21<sup>st</sup> 2026, 14:00**

### 7.1 MNIST Classification with a Neural Network

Recall the convolutional neural network setup. In this exercise, a model predicts class labels for a set of handwritten digits. Before, we saw the binary classification with two classes and solved it with a logistic regression. In our case, however, the amount of classes is 10, so the model we will use is a multinomial logistic regression (also called softmax regression). The output of the model is a probability distribution consisting of 10 bounded reals.

#### Neural network architecture

- A convolutional layer with a single input channel, 32 output channels, a kernel size of  $(3, 3)$ , a stride of one, and a ReLU activation.
- A 2-d max-pooling layer with a kernel size of  $(2, 2)$ .
- A convolutional layer with 32 input channels, 64 output channels, a kernel size of  $(3, 3)$ , a stride of one, and a ReLU activation.
- A 2-d max-pooling layer with a kernel size of  $(2, 2)$ .
- A dropout layer with a dropout ratio of 20 percent.
- A fully connected layer with 1600 inputs and 128 outputs including a ReLU activation.
- A dropout layer with a dropout ratio of 20 percent.
- An output layer with 128 inputs and 10 outputs. Note: use `nn.CrossEntropyLoss` as your loss function, which combines a softmax activation and the negative log-likelihood loss in a single, numerically stable operation. Do not apply a softmax activation inside your `forward()` method.

## Exercise

A coding template is provided for this exercise — you are encouraged to use it as a starting point, but you are also welcome to implement everything from scratch if you prefer. The following packages are required: `torch`, `torchvision`, `matplotlib`, `tqdm`, `numpy`.

**Note on hardware:** Training can be accelerated by passing `--device cuda` to the script. This requires a CUDA-capable GPU and the appropriate CUDA toolkit to be installed. Training on the CPU is perfectly fine for this exercise and requires no additional setup.

Be aware that the model training process is time-consuming.

- Implement the neural network architecture as stated above.
- Train the model and validate it on the validation set. Collect values for training loss, validation loss, training accuracy, and validation accuracy for each epoch and plot them. How do they change over epochs? What might occur in the case of many training epochs?
- With the `classify` function you can plot a subset of misclassified digits, cf. Figure 1. Do you notice any particular problems when looking at your classification results?
- What is the purpose of a softmax activation?

## 7.2 Ablation Studies

Ablation studies are a systematic method for determining the importance and relevance of individual elements in a neural network architecture for a given task. This is achieved by selectively removing specific parts and observing how the network performs without them.

For this task, we will make some amendments to the network. Please make a copy of the network, apply the changes to your copied version and then train it.

For each run, note down: Which changes have you made to the network? What are the effects, e.g. on training results, network performance and training evolution over several epochs, compared to the original network?

Try some of the following ideas (or feel free to make your own changes):

- Remove the dropout layers. Does this have an impact on the training procedure? What does a dropout layer do and when does it influence the functioning of a neural network?

- Remove the Max Pooling layers and check the effect on the training results. What does a Max Pooling layer do? You can also try to replace it with an Average Pooling layer.
- Remove or replace the ReLU activation layers. Does your network still perform well?
- Replace `CrossEntropyLoss` with `MSELoss`. To do so, you will need to one-hot encode the target labels before computing the loss. Does the network still converge? How does the training behavior — e.g. loss curves and convergence speed — compare to the original? Why is cross-entropy generally preferred over MSE for classification tasks?
- Decrease or increase the number of convolutional or fully connected layers.
- Introduce batch normalization layers - do they have an impact on your training or the classification results?
- (*More advanced*): residual skip connections have been designed for the training of deep neural networks. Look them up and implement them yourself. Can you recognize any impact on our example? What is their original purpose?

Note that some changes have an impact on the number of output channels, so you may have to manually compute and adjust the input channels of a subsequent layer.

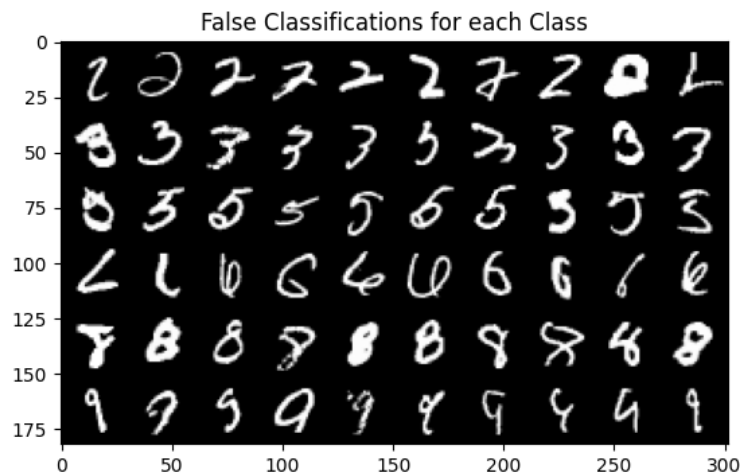


Figure 1: A plot of wrongly classified handwritten digits. Each row depicts false classifications for a specific class.